# Day 2: Budget Alert + IAM Users, Groups, Policies, Roles + AWS CLI

Sunday, 11 October 2026, 7:00-9:00 PM IST · Trainer: Gangadhar Ure

---

Live class: Sunday 11 Oct 2026, 7:00-9:00 PM IST
Exam focus: Domain 2 (Security and Compliance), Domain 4 (Billing)
Exam tasks: 2.3 · 3.1 · 4.2

Before class: root MFA and the IAM user `Cloud-Unlock` from Day 1 are done. If you got stuck, bring a screenshot and we will help.

Regions, Availability Zones and edge locations were taught on Day 1. Shared responsibility comes on Day 3.

### Practice Sequence

| Seq | Practice | File |
|---|---|---|
| 04 | Budget alert, `AdminGroup`, MFA on `Cloud-Unlock`, sign in as `Cloud-Unlock` | 04-account-security-lab.md (Labs 2 and 4) |
| 05 | IAM basics: users, groups, policies, roles, least privilege | 05-iam-basics.md |
| 06 | IAM hands-on: read-only groups, allowed vs denied, AWS CLI in CloudShell, custom policy, switch role | 06-iam-hands-on-lab.md |

**Live class:** keep practical-steps.md open and tick each step with the trainer.

### By The End Of Today

- You have a budget alert and know that budgets alert, they do not stop spending.
- Your admin access comes from `AdminGroup`, your IAM user has MFA, and you work as `Cloud-Unlock` (not root).
- You can explain IAM users, groups, policies and roles in one line each.
- You can read a simple IAM policy: `Effect`, `Action`, `Resource`.
- You practised least privilege: one allowed action and one `Access Denied`.
- You switched into a read-only role and back.
- You ran AWS CLI commands in CloudShell (no access keys) and created and deleted an S3 bucket.

### Today's Deliverables

- Screenshot of the budget list
- Screenshot of `AdminGroup` with `Cloud-Unlock` (account ID hidden)
- Screenshot of one allowed action and one `Access Denied`

- `custom-ec2-describe-only.json` in `code-or-policies/`
- Screenshot of your CLI output (masked ARN and IAM lists)
- Day 2 LinkedIn post with `#CloudUnlock`: see 10-linkedin-post.md

### After Day 2

1. Optional: 07-optional-challenge.md
2. Clean up: 08-cleanup.md
3. Submit: 09-submission-format.md
4. Revise: 11-quiz-prep.md

# Day 2 - IAM Basics

Goal: understand how AWS decides who can do what.

Exam tasks: 2.3 (AWS access management), 2.1 (shared responsibility: access is your job)

Remember this:

> *Identity + Permissions = Access*

## The Story

Think of your **college ERP**. A student sees only their own marks. Faculty can enter marks. The admin manages everything. Same app, different access. That is what IAM does for your AWS account.

## IAM In Simple Words

| IAM word | One line | College ERP example |
|---|---|---|
| **User** | One person with a login | You, with your student login |
| **Group** | A team of users with the same access | All faculty |
| **Policy** | JSON rules that allow or deny actions | "Faculty can edit marks" |
| **Role** | Temporary access for a task, no password | A visitor pass that expires |
| **Root user** | The sign-up email, full access | The college owner's master key |

Login access does not mean full access. **Policies decide what a user can do.**

## IAM Users

- A named identity for one person (or an application).
- Signs in to the Console with a password, and can have MFA.
- Example from Day 1: `Cloud-Unlock` is your daily admin user.

## IAM Groups

- A collection of users with common permissions.
- Attach the policy to the **group**, then add users to the group.
- A group cannot contain another group.

Examples we use today:

- `AdminGroup` uses `AdministratorAccess`.
- `S3ReadOnlyGroup` uses `AmazonS3ReadOnlyAccess`.
- `S3FullAccessGroup` uses `AmazonS3FullAccess`.

Exam pointer: "manage permissions for many users" = **IAM groups**.

### IAM Policies

An IAM policy is a JSON document. It has three main words:

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": ["s3:Get*", "s3:List*"], "Resource": "*"
}
  ]
}
```

| Word | Meaning |
|---|---|
| `Effect` | `Allow` or `Deny` |
| `Action` | What can be done, for example `s3:ListBucket` |
| `Resource` | On which resource, `*` means all |

Types:

- **AWS managed policy:** created and maintained by AWS, for example `AmazonS3ReadOnlyAccess`.
- **Customer managed policy:** created by you and reusable.
- **Inline policy:** attached directly to one user, group or role.

Rule: an explicit **Deny** always wins over an Allow.

### IAM Roles

- A role gives **temporary** credentials. No password, no long-term keys.
- Used when:
    - an AWS service needs to call another AWS service (for example, an EC2 server reading S3);
    - a person switches into a role for a task.

Exam pointer: "an application on EC2 needs to access S3" = **IAM role**, not access keys in code.

### Least Privilege

Give only the permissions needed for the task.

- Needs to view S3 only → `AmazonS3ReadOnlyAccess`, not `AmazonS3FullAccess`.
- A Myntra support agent can see an order but cannot delete the catalogue.

### Keeping Access Safe

- **MFA** on the root user and on every IAM user.

- **Access keys** are for the CLI or code. Never share them, never put them on GitHub. Never create them for root.
- **Password policy:** set a minimum length and require numbers and symbols.
- Do daily work as an IAM user. Use root only for root-only tasks.

## Official Links

| Topic | Link |
|---|---|
| What is IAM | https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html |
| IAM best practices | https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html |
| IAM groups | https://docs.aws.amazon.com/IAM/latest/UserGuide/id_groups_create.html |
| Policies and permissions | https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies.html |
| IAM roles | https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html |

## Day 2 - IAM Hands-On Lab

Today, practise least privilege using IAM users, groups, policies and a role, then use the AWS CLI in CloudShell.

Exam tasks: 2.3 (AWS access management), 3.1 (ways to access AWS: Console, CLI, SDK, IaC)

### Practice Rule

Create only read-only access in this lab. Do not give full access unless asked. Do Labs 2 and 4 of 04-account-security-lab.md first (budget and `AdminGroup`), then work as `Cloud-Unlock`.

Practice in this order:

1. Create group.
2. Attach read-only policy to group.
3. Create user.
4. Add user to group.
5. Test allowed access.
6. Test denied access.

### Lab 1 - S3 Read-Only Access

Create:

- Group: `S3ReadOnlyGroup`
- Policy: `AmazonS3ReadOnlyAccess`
- User: `Cloud-Unlock-RUser`

Steps (as `Cloud-Unlock`):

1. **IAM → User groups → Create group →** `S3ReadOnlyGroup` → attach `AmazonS3ReadOnlyAccess` → **Create user group**.
2. **IAM → Users → Create user →** `Cloud-Unlock-RUser` → tick **Provide user access to the AWS Management Console → Custom password → Next**.
3. Permissions options: **Add user to group** → tick `S3ReadOnlyGroup` → **Next → Create user**.

Test:

- Open a private or incognito window. Sign in with your alias sign-in URL as `Cloud-Unlock-RUser`.
- Open **S3**. The bucket list opens (**allowed**).
- Click **Create bucket** and try to create one. It fails (**denied**).
- Capture the `Access Denied` message.

### Lab 2 - S3 Write Access

Create:

- Group: `S3FullAccessGroup`
- Policy: `AmazonS3FullAccess`
- User: `Cloud-Unlock-WUser`

Test:

- Sign in as `Cloud-Unlock-WUser`.
- **Create bucket** `cloud-unlock-<yourname>-wuser-test` (Region Mumbai, defaults). It **works** (**allowed**).
- Same action, two users: `Cloud-Unlock-RUser` is denied, `Cloud-Unlock-WUser` is allowed. Policies decide.
- Delete the bucket right away: select it → **Delete** → type the name → **Delete bucket**.

### Lab 3 - Read A Policy (JSON)

1. **IAM → Policies** → search `AmazonS3ReadOnlyAccess` → open it → **JSON** tab.
2. Find `Effect`, `Action` and `Resource`.
3. Write one line in `notes.md`: what does this policy allow?

### Lab 4 - Custom Policy And A Role (Switch Role)

Create a customer managed policy:

> `CustomEC2DescribeOnly`

1. **IAM → Policies → Create policy → JSON** → paste:

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": ["ec2:Describe*"], "Resource": "*" }
  ]
}
```

2. **Next** → name `CustomEC2DescribeOnly` → **Create policy**.

Create a role:

- Role: `TrainingReadOnlyRole`
- Trusted entity: **AWS account → This account**
- Policy: `ReadOnlyAccess`

3. **IAM → Roles → Create role → AWS account → This account → Next →** tick `ReadOnlyAccess` **→ Next →** name `TrainingReadOnlyRole` **→ Create role**.

Test (as `Cloud-Unlock`):

- Top right account menu → **Switch role**.
- Account: your account alias. Role: `TrainingReadOnlyRole`. → **Switch role**.
- The top right now shows the role. Try to create anything: it fails (**read-only**).
- Account menu → **Switch back**.

Remember: a role gives **temporary** access. Same person, different hat.

### Lab 5 - Password Policy

1. **IAM → Account settings → Password policy → Edit**.
2. Choose **Custom**: minimum length `12`, require at least one number and one non-alphanumeric character.
3. **Save changes**.

### Lab 6 - AWS CLI In CloudShell

The CLI is one of the **four ways to access AWS**: Console (click), CLI (type), SDK (code), Infrastructure as Code (template). All of them call the same AWS APIs.

On your own laptop, `aws configure` asks for an **Access key ID**, **Secret access key**, **Region** and **output format**. Access keys are for the CLI and code only: never share them, never put them on GitHub, never create them for root. Today we use **CloudShell**, which needs no keys.

Run in CloudShell (Region Mumbai), signed in as `Cloud-Unlock`:

```
aws sts get-caller-identity --query Arn --output text | sed -E 's/[0-9]
{12}/XXXXXXXXXXXX/'
aws iam list-users --query "Users[].UserName" --output table
aws iam list-attached-user-policies --user-name Cloud-Unlock
aws s3 mb s3://cloud-unlock-<yourname>-cli-demo --region ap-south-1
echo "Hello from Cloud Unlock" > hello.txt
aws s3 cp hello.txt s3://cloud-unlock-<yourname>-cli-demo/
aws s3 ls s3://cloud-unlock-<yourname>-cli-demo/
aws s3 rb s3://cloud-unlock-<yourname>-cli-demo --force
```

Expected:

- The masked ARN ends with `user/Cloud-Unlock`. Never share the unmasked output; it shows your account ID.
- `list-attached-user-policies` is empty, because access comes from `AdminGroup`.
- The bucket is created, the file is listed, then the bucket is removed.

### Add These In Your Submission

- Screenshot of `S3ReadOnlyGroup` with `Cloud-Unlock-RUser` (account ID hidden)
- Screenshot of one **allowed** and one **Access Denied**
- `code-or-policies/custom-ec2-describe-only.json` with your policy
- Screenshot of the role after **Switch role** (account ID hidden)
- Screenshot of your CLI output (masked ARN and IAM lists)

### If You Get Stuck

| Problem | Try this |
|---|---|
| Can't find the sign-in URL for `Cloud-Unlock-RUser` | **IAM → Dashboard** shows the account sign-in URL (or your alias URL) |
| Switch role says "invalid information" | Check the account alias and the exact role name `TrainingReadOnlyRole` |
| `Cloud-Unlock-RUser` must change password at first sign-in | Normal if that box was ticked. Set a new strong password |
| You can still do everything as `Cloud-Unlock-RUser` | Check the user is only in `S3ReadOnlyGroup` and has no other policy |

Submit the part you completed. Add a note like:

```
I completed S3 read-only access. I got stuck while testing switch role.
```

### Cleanup (end of the week)

Delete `Cloud-Unlock-RUser`, `Cloud-Unlock-WUser`, `S3ReadOnlyGroup`, `S3FullAccessGroup`, `CustomEC2DescribeOnly`, `TrainingReadOnlyRole` and any `cli-demo` bucket when you finish the week. **Keep** `Cloud-Unlock` and `AdminGroup`.

### Official Links

| Topic | Link |
|---|---|
| Create an IAM user | https://docs.aws.amazon.com/IAM/latest/UserGuide/id_users_create.html |
| Create IAM groups | https://docs.aws.amazon.com/IAM/latest/UserGuide/id_groups_create.html |
| Create a policy (JSON) | https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_create-console.html |
| Switch to a role in the console | https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_use_switch-role-console.html |
| Password policy | https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_passwords_account-policy.html |

| Topic | Link |
|---|---|
| AWS CloudShell | https://docs.aws.amazon.com/cloudshell/latest/userguide/welcome.html |
| AWS CLI user guide | https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-welcome.html |